

Lenguaje PL/SQL Oracle – Parte 4

Los Disparadores – Los Triggers

En PL/SQL, los disparadores (triggers) son objetos de base de datos que se ejecutan automáticamente en respuesta a eventos específicos en una tabla, como inserciones, actualizaciones o eliminaciones de datos. Los disparadores son pequeños fragmentos de código que no se ejecutan directamente; simplemente "ocurren" como efecto secundario de alguna otra operación, no se los invoca, suceden. Debido a que funcionan como un efecto secundario, la gente a menudo olvida que existen.

Un trigger es un bloque de código PL/SQL asociado a una tabla que se dispara cuando ocurre un evento específico en esa tabla.

Un disparador o desencadenador (trigger) es un bloque de código que se ejecuta automáticamente cuando ocurre algún evento (como inserción, actualización o borrado) sobre una determinada tabla (o vista); es decir, cuando se intenta modificar los datos de una tabla (o vista) asociada al disparador.

Existen distintos tipos de eventos, los triggers pueden activarse antes o después de operaciones DML (Data Manipulation Language), como INSERT, UPDATE o DELETE.

Los disparadores tienen como objetivo:

- **Integridad de datos:** Validar datos antes de insertarlos o actualizar, o tomar acciones si una operación viola las reglas de la base de datos a fin de mantener la integridad referencial.
- **Coherencia de datos:** Conservar la relación entre los datos entre distintas tablas.
- **Las Reglas de Negocio:** Ejecutar reglas de negocio complejas sin dejarlas en manos de los programadores de las aplicaciones.
- **Auditaría:** Registrar cambios en los datos sobre las tablas y la identidad de quien los realizó, como quién los realizó y cuándo lo hizo.
- **Automatización:** Realizar acciones automáticamente en respuesta a eventos, como enviar notificaciones o actualizar otras tablas.
- **Otros:** realizar cualquier acción cuando una tabla es modificada.

Por ejemplo podemos crear un disparador para que:

- impida la inserción de un registro con un valor duplicado en una columna clave.
- registre en otra tabla cada vez que se actualiza una fila en una tabla específica.
- envíe un correo electrónico cuando se realiza una venta.

Si se intenta modificar (agregar, actualizar o eliminar) datos de una tabla asociada a un disparador, el disparador se ejecuta (se dispara) en forma automática.

La diferencia con los procedimientos almacenados del sistema es que los disparadores:

- No pueden ser invocados directamente, el disparador se ejecuta automáticamente. Por ejemplo, al intentar modificar los datos de una tabla asociada a un disparador, el disparador se ejecuta en forma automática.
- No reciben y retornan parámetros.
- Son apropiados para mantener la integridad de los datos y cumplir con ciertas reglas de negocio, no para obtener resultados de consultas. Por ejemplo, no permitir que en días no laborales los datos de las tablas sean modificados.

Los triggers son objetos de la base de datos, por ende sus nombres cumplirán la misma regla que cualquier otro tipo de identificador. Para obtener información de los disparadores se puede consultar el diccionario de la Base de Datos:

La tabla **user_objects** nos muestra todos los objetos de la base de datos seleccionada, incluidos los disparadores. En la columna **object_type** aparece **trigger** si es un disparador. En el siguiente ejemplo solicitamos todos los objetos que son disparadores:

```
SELECT *FROM user_objects WHERE OBJECT_TYPE='trigger';
```

La tabla **user_triggers** nos muestra todos los disparadores de la base de datos actual. Muestra el nombre del desencadenador (trigger_name), si es before o after y si es a nivel de sentencia o por fila (trigger_type), el evento que lo desencadena (triggering_event), a qué objeto está asociado, si tabla o vista (base_object_type), el nombre de la tabla al que está asociado (table_name), los campos, si hay referencias, el estado, la descripción, el cuerpo (trigger_body), etc.

```
SELECT trigger_name, triggering_event FROM user_triggers WHERE trigger_name LIKE 'TR%';
```

Los disparadores se ejecutarán si se encuentran habilitados. Los triggers se pueden habilitar o deshabilitar. Podemos saber si un trigger está o no habilitado consultando en el diccionario de la Base de Datos la tabla **user_triggers**. En dicha consulta, en la columna **status** aparecerá **enabled** si está habilitado o **disabled** si está deshabilitado. Se pueden visualizar todos los triggers del usuario o uno en particular:

```
SELECT status FROM user_triggers;
```

```
SELECT status FROM user_triggers WHERE TRIGGER_NAME = 'nombre_del_trigger';
```

Mediante la tabla **user_source** se puede visualizar el código fuente almacenado en un disparador. En el siguiente ejemplo solicitamos el código fuente del objeto "tr_insertar_libros":

```
SELECT *FROM user_source WHERE name='TR_INSERTAR_LIBROS';
```

Sintaxis de un Disparador

La sintaxis general para crear un disparador es la siguiente:

```
Cabecera + Momento + Evento + Campos + Tabla + Nivel + Condición + Bloque PL/SQL
```

En la cabecera irá:

```
CREATE OR REPLACE TRIGGER nombre_disparador
```

Los disparadores se crean con la instrucción **CREATE TRIGGER** seguido del nombre del disparador. Si se agrega **OR REPLACE**, al momento de crearlo si ya existe un disparador con el mismo nombre, tal disparador será reemplazado por el que se está creando.

Luego se define en qué **momento** se disparará:

```
BEFORE, AFTER o INSTEAD OF
```

El momento indica cuándo se ejecutará, disparará, el disparador en relación al evento indicado. Este puede ser BEFORE (antes), AFTER (después) o INSTEAD OF (en lugar de). **BEFORE** significa que el disparador se activará antes de que se ejecute la operación **insert**, **update** o **delete** sobre la tabla, que causó el disparo de este. **AFTER** significa que el disparador se activará después que se ejecute la operación que causó el disparo. **INSTEAD OF** sólo puede definirse sobre vistas, anula la sentencia disparadora, se ejecuta en lugar de tal sentencia (ni antes ni después).

Debemos indicar cuál es el **evento** que disparara el trigger:

INSERT, UPDATE o DELETE

El **evento** especifica la operación, acción, tipo de modificación, que causa que el disparador se active, puede ser **INSERT**, **UPDATE** o **DELETE**. Debe colocarse al menos una acción, puede ser más de una, en tal caso se separan con **OR**. Si la acción es **UPDATE** lleva una lista de atributos, el disparador sólo se ejecuta si se actualiza algún atributo de la lista.

En caso de tener un disparador para actualizaciones, **UPDATE** se deben dar la lista de campos:

OF campos

Se indica sobre qué tabla actuará el disparador

ON nombretabla

ON nombretabla indica la tabla o vista asociada al disparador.

Se dará el **nivel** ya sea de sentencia o de fila:

STATEMENT o FOR EACH ROW

El disparador puede ser a **nivel de sentencia** o a **nivel de fila**. **FOR EACH ROW** indica que el disparador es a nivel de fila, es decir, se activa una vez por cada registro afectado por la operación sobre la tabla, cuando una sola operación afecta a varios registros. Los disparadores a nivel de sentencia se activan una sola vez, antes o después de ejecutar la operación sobre la tabla. Si no se especifica, o se especifica **STATEMENT**, es a nivel de sentencia.

Se puede declarar una **condición**, esto es opcional:

WHEN condicion

La cláusula **WHEN** en la definición de un disparador permite establecer una condición que debe ser verdadera para que el disparador se ejecute. Si la condición **WHEN** es falsa, el disparador no se activa, y la acción definida en su cuerpo no se realiza.

Luego se debe dar el código del disparador, qué hará cuando se active, este es un bloque PL/SQL, llamado el cuerpo del disparador:

```
BEGIN
-- cuerpo del disparador, sentencias;
END nombre_disparador;
/
```

El cuerpo del disparador son las acciones que se ejecutan al dispararse el trigger, las condiciones que determinan cuando un intento de inserción, actualización o eliminación provoca las acciones que el trigger realizará. El bloque se delimita con **BEGIN... END; /**.

Entonces, un disparador es un bloque de código asociado a una tabla que se dispara automáticamente antes o después de una sentencia **INSERT**, **UPDATE** o **DELETE** sobre una tabla determinada.

Consideraciones Generales

Tener en cuenta las siguientes consideraciones:

- No están permitidas en un desencadenador, las siguientes instrucciones:
 - CREATE DATABASE,
 - ALTER DATABASE,
 - DROP DATABASE,
 - LOAD DATABASE,

- RESTORE DATABASE,
 - LOAD LOG,
 - RECONFIGURE,
 - RESTORE LOG,
 - DISK INIT,
 - DISK RESIZE.
- Se pueden crear varios disparadores para cada evento, es decir, para cada tipo de modificación, inserción, actualización o borrado, para una misma tabla. Por ejemplo, se puede crear un disparador **INSERT** para una tabla que ya tiene otro disparador **INSERT**, o que ya posee un disparador **DELETE**.

Disparadores de Inserción a Nivel de Sentencia

Hemos visto que un disparador está asociado a una tabla y a una operación específica sobre ella: inserción, actualización o eliminación de registros. Veamos como realizar la creación de un disparador para el evento de inserción: disparador de inserción.

La siguiente es la sintaxis para crear un trigger de inserción que se dispare cada vez que se ejecute una sentencia "insert" sobre la tabla especificada, es decir, cada vez que se intenten ingresar datos en la tabla:

```
CREATE OR REPLACE TRIGGER nombredisparador
Momento INSERT
ON nombretabla
BEGIN
    cuerpo del trigger;
END nombredisparador;
/
```

Luego de la orden **CREATE TRIGGER** se coloca el nombre del disparador. Si se agrega **OR REPLACE** si al momento de crearlo ya existe un disparador con el mismo nombre, la definición del nuevo disparador reemplazará la definición del anterior, o sea que el anterior disparador quedará como "borrado" y vuelto a crear.

El **Momento** indica cuando se disparará el trigger en relación con el evento: puede ser **BEFORE** (antes) o **AFTER** (después). Se especifica el evento que causa que el trigger se dispare, en este caso **INSERT**, ya que el trigger se activará cada vez que se ejecute la sentencia **INSERT** sobre la tabla especificada luego de **ON**.

Es un trigger a nivel de sentencia, es decir, se dispara una sola vez por cada sentencia **INSERT**, aunque la sentencia ingrese varios registros. Es el valor por defecto si no se especifica. Finalmente se coloca el cuerpo del trigger dentro del bloque **BEGIN... END**.

Las siguientes sentencias dispararán un disparador de inserción:

```
INSERT INTO tabla VALUES ...;
INSERT INTO tabla SELECT ... FROM...;
```

Para identificar fácilmente los disparadores de otros objetos se recomienda usar un prefijo y darles el nombre de la tabla para la cual se crean junto al tipo de acción. En caso de que una sola sentencia **INSERT** ingrese varios registros en la tabla asociada, el trigger se disparará una sola vez, si se desea que se active una vez por cada registro afectado, debemos indicarlo con **FOR EACH ROW**.

Disparadores de Inserción a Nivel Fila

La siguiente es la sintaxis para crear un trigger de inserción a nivel de fila, se disparará una vez por cada fila ingresada sobre la tabla especificada:

```
CREATE OR REPLACE TRIGGER nombredisparador
momento INSERT
ON nombretabla
FOR EACH ROW
BEGIN
    cuerpo del trigger;
END nombredisparador;
/
```

Disparadores de Eliminación a Nivel Fila

Un disparador está asociado a una tabla y a una operación específica (inserción, actualización o eliminación). Para crear un disparador para el evento de borrado: **DELETE TRIGGER**, de eliminación, o sea que se dispare cada vez que se ejecute una sentencia **DELETE** sobre la tabla específica, es decir, cada vez que se eliminen registros de la tabla, debemos utilizar la siguiente sintaxis:

```
CREATE OR REPLACE TRIGGER nombredisparador
momento DELETE
ON nombretabla
nivel-- STATEMENT O FOR EACH ROW
BEGIN
    cuerpo del trigger;
END nombredisparador;
/
```

Luego de la instrucción **CREATE TRIGGER** o **CREATE OR REPLACE TRIGGER** se coloca el nombre del disparador. El **momento** indica cuándo se disparará el trigger en relación con el evento, puede ser **BEFORE** (antes) o **AFTER** (después). Se especifica el evento que causa que el trigger se dispare, en este caso **DELETE**, ya que el trigger se activará cada vez que se ejecute la sentencia **DELETE** sobre la tabla especificada luego de **ON**. En **nivel** se especifica si será un trigger a nivel de **sentencia** (se dispara una sola vez por cada sentencia **delete**, aunque la sentencia elimine varios registros) o a nivel de **fila** (se dispara tantas veces como filas se eliminan en la sentencia **delete**). Finalmente se coloca el cuerpo del trigger dentro del bloque **BEGIN... END**.

Disparador de Actualización a Nivel de Sentencia

Un disparador para el evento de actualización, **UPDATE TRIGGER**, a nivel de sentencia lleva la siguiente es la sintaxis:

```
CREATE OR REPLACE TRIGGER nombredisparador
momento UPDATE
ON nombretabla
STATEMENT
BEGIN
    cuerpo del trigger;
END nombredisparador;
/
```


Para crear un trigger de actualización, que se dispare cada vez que se ejecute una sentencia **UPDATE** sobre la tabla especificada, o sea cada vez que se intenten modificar datos en la tabla, luego de la instrucción **CREATE OR REPLACE TRIGGER** se coloca el nombre del disparador. El **momento** indica cuándo se disparará el trigger en relación con el evento, puede ser **BEFORE** (antes) o **AFTER** (después). Se especifica el evento que causa que el trigger se dispare, en este caso **UPDATE**, ya que el trigger se activará cada vez que se ejecute la sentencia **UPDATE** sobre la tabla especificada luego de **ON**.

STATEMENT significa que es un trigger a nivel de sentencia, es decir, se dispara una sola vez por cada sentencia **UPDATE**, aunque la sentencia modifique varios registros; como en cualquier trigger, si no se especifica es el valor por defecto. Finalmente se coloca el cuerpo del trigger dentro del bloque **BEGIN... END**. Las siguientes sentencias disparan un trigger de inserción:

```
UPDATE tabla SET campo=nuevovalor;  
UPDATE tabla SET campo=nuevovalor WHERE condicion;
```

Vimos la creación de un disparador para el evento de actualización a nivel de sentencia, es decir, se dispara una vez por cada sentencia **UPDATE** sobre la tabla asociada. En caso de que una sola sentencia **UPDATE** modifique varios registros en la tabla asociada, el trigger se disparará una sola vez;

Disparador de Actualización a Nivel de Fila

Si se desea que se active un disparador cada vez que un registro es actualizado, debemos indicarlo con **FOR EACH ROW**.

La siguiente es la sintaxis nos permite crear un trigger de actualización a nivel de fila, se dispare una vez por cada fila modificada sobre la tabla especificada:

```
CREATE OR REPLACE TRIGGER nombredisparador  
momento UPDATE  
ON nombretabla  
FOR EACH ROW  
BEGIN  
    cuerpo del trigger;  
END nombredisparador;  
/
```

El trigger de actualización (a nivel de sentencia o de fila) permite incluir una lista de campos. Si se incluye el nombre de un campo (o varios) luego de "update", el trigger se disparará únicamente cuando alguno de esos campos (incluidos en la lista) es actualizado. Si se omite la lista de campos, el trigger se dispara cuando cualquier campo de la tabla asociada es modificado, es decir, por defecto toma todos los campos de la tabla.

Tengamos en cuenta que la lista de campos solamente puede especificarse en disparadores de actualización, nunca en disparadores de inserción ni de eliminación:

```
CREATE OR REPLACE TRIGGER nombredisparador  
momento UPDATE OF campos  
ON tabla  
nivel--STATEMENT O FOR EACH ROW  
BEGIN  
    cuerpodel disparador;  
END nombredisparador;  
/
```

Campos son los campos de la tabla asociada que activarán el trigger si son modificados. Pueden incluirse más de uno, en tal caso, se separan con comas.

Creación de Disparadores de Múltiples Eventos

Hasta el momento hemos aprendido a crear un disparador (trigger) asociado a una única operación (inserción, actualización o borrado). Pero un trigger puede definirse sobre más de un evento; en tal caso se separan con **OR**.

La sintaxis para crear un disparador para múltiples eventos es la siguiente:

```
CREATE OR REPLACE TRIGGER nombredisparador
momento-- BEFORE, AFTER O INSTEAD OF
OF campo--SI ALGUNO DE LOS EVENTOS ES UPDATE
eventos-- INSERT, UPDATE Y/O DELETE
ON nombretabla
nivel--PUEDEN SER A NIVEL DE SENTENCIA (STATEMENT) O DE FILA (FOR EACH ROW)
BEGIN
    cuerpo del disparador--SENTENCIAS
END nombredisparador;
/
```

Si el disparador se define para más de un evento desencadenante, en el cuerpo de éste se puede emplear un condicional para controlar cuál operación disparó el trigger. Esto permite ejecutar bloques de código según la clase de acción que disparó el desencadenador.

Para identificar el tipo de operación que disparó el trigger empleamos las cláusulas **INSERTING**, **UPDATING** y **DELETING**.

Veamos esto con un ejemplo. Se va a definir trigger a nivel de sentencia, para los eventos **INSERT**, **UPDATE** y **DELETE**; sobre los datos de la tabla **libros**, que almacenará en la tabla **control** el nombre del usuario, la fecha y el tipo de modificación que alteró la tabla, con la siguiente leyenda dependiendo de la acción que disparó en trigger:

- si se realizó una inserción (**insert**), se almacena "inserción";
- si se realizó una actualización (**update**), se almacena "actualización" y
- si se realizó una eliminación (**delete**) se almacena "borrado".

```
CREATE OR REPLACE TRIGGER tr_cambios_libros
BEFORE INSERT OR UPDATE OR DELETE
ON libros
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO control VALUES (USER, SYSDATE, 'INSERCIÓN');
    END IF;
    IF UPDATING THEN
        INSERT INTO control VALUES (USER, SYSDATE, 'ACTUALIZACIÓN');
    END IF;
    IF DELETING THEN
        INSERT INTO control VALUES (USER, SYSDATE, 'BORRADO');
    END IF;
END tr_cambios_libros;
/
```

Si ejecutamos un **INSERT** sobre libros, el disparador se activa entrando por el primer **IF**; si ejecutamos un **UPDATE** el trigger se dispara entrando por el segundo **IF**; si ejecutamos un **DELETE** el desencadenador se dispara entrando por el tercer **IF**.

Las siguientes sentencias disparan el trigger creado anteriormente:

```
INSERT INTO tabla VALUES ...;  
INSERT INTO tabla SELECT ... FROM...;  
UPDATE tabla SET...;  
DELETE FROM tabla...;
```

Cuando trabajamos con disparadores a nivel de fila, Oracle provee de dos tablas temporales con las que se puede trabajar, que contienen los valores de los campos del registro afectado por la sentencia que disparó el trigger tanto antiguos como nuevos (los valores antes de ser actualizados y los nuevos después de ser actualizados. El nuevo valor es **:NEW** y el viejo valor es **:OLD**. Para referirnos a ellos debemos especificar su campo separado por un punto **:NEW.CAMPO** y **:OLD.CAMPO**.

El acceso a estos campos depende del evento del disparador:

- En un trigger disparado por un **INSERT**, se puede acceder al campo **:NEW** únicamente, el campo **:OLD** está vacío, contiene **NULL**.
- En una inserción se puede emplear **:NEW** para escribir nuevos valores en las columnas de la tabla.
- En un trigger que se dispara con **UPDATE**, se puede acceder a ambos campos. En una actualización, se pueden comparar los valores de **:NEW** y **:OLD**.
- En un trigger de eliminación, únicamente se puede acceder al campo **:OLD**, ya que el campo **:NEW** no existe luego que el registro es eliminado, el campo **:NEW** contiene **NULL** y no puede ser modificado.
- Los valores de **:OLD** y **:NEW** están disponibles en triggers **AFTER** y **BEFORE**.
- El valor de **:NEW** puede modificarse en un trigger **BEFORE**, es decir, se puede acceder a los nuevos valores antes que se ingresen en la tabla y cambiar los valores asignando a **:NEW.campo** otro valor.
- El valor de **:NEW** no puede modificarse en un trigger **AFTER**, esto es porque el trigger se activa luego que los valores de **:NEW** se almacenen en la tabla.
- El campo **:OLD** nunca se lo puede modificar, sólo puede leerse.
- Pueden usarse en una cláusula **WHEN**.
- En el cuerpo el trigger, los campos **OLD** y **NEW** deben estar precedidos por : (dos puntos), pero si está en **WHEN** no.

Tener en cuenta que:

- Sólo se puede acceder a los valores de **:NEW** y **:OLD** en disparadores a nivel de fila, no en disparadores a nivel de sentencia.
- Están disponibles **:NEW** y **:OLD** en triggers **AFTER** y **BEFORE**.
- En los triggers a nivel de fila, se puede incluir una restricción adicional, agregando la cláusula **WHEN** con una condición que se evalúa para cada fila que afecte el disparador; si resulta cierta, se ejecutan las sentencias del trigger para ese registro; si resulta falsa, el trigger no se dispara para ese registro.

Limitaciones de WHEN

La cláusula WHEN:

- no puede contener subconsultas, funciones agregadas ni funciones definidas por el usuario;
- sólo se puede hacer referencia a los parámetros del evento;
- no se puede especificar en los triggers **INSTEAD OF** ni en trigger a nivel de **sentencia**.

Por otro lado, en un trigger de actualización a nivel de fila, se puede especificar el nombre de un campo en la condición **UPDATING** para determinar cuál campo ha sido actualizado.

Sintaxis básica:

```
CREATE OR REPLACE TRIGGER nombretigger
momento UPDATE...
FOR EACH ROW
BEGIN
  IF UPDATING ('campo') THEN
    ...
  END IF;
END nombredisparador;
/
```

Disparador de Actualización - Campos (UPDATING)

El siguiente trigger se dispara cuando se actualiza la tabla **libros**.

```
CREATE OR REPLACE TRIGGER tr_actualizar_libros BEFORE UPDATE ON libros
FOR EACH ROW
BEGIN
  IF updating('precio') THEN
    INSERT INTO controlprecios VALUES (sysdate, :old.codigo, :old.precio, :new.precio);
  ELSE
    INSERT INTO control VALUES (user, sysdate, :old.codigo);
  END IF;
END tr_actualizar_libros;
/
```

Dentro del cuerpo del trigger se averigua qué campo ha sido modificado; en caso de modificarse el **precio**, se ingresa en la tabla **controlPrecios** la fecha, el código del libro y el antiguo y nuevo precio; en caso de actualizarse cualquier otro campo, se almacena en la tabla **control** el nombre del usuario que realizó la modificación, la fecha y el código del libro modificado.

Habilitar y Deshabilitar Disparadores

Un disparador puede estar en dos estados: habilitado (enabled) o deshabilitado (disabled). Cuando se crea un trigger, por defecto está habilitado.

Se puede deshabilitar un trigger para que no se ejecute. Un trigger deshabilitado sigue existiendo, pero al ejecutar una instrucción que lo dispare, no se activa.

Sintaxis para deshabilitar un trigger:

```
ALTER TRIGGER nombredisparador DISABLE;
```

Ejemplo: Deshabilitamos el trigger **tr_ingresar_empleados**:

```
ALTER TRIGGER TR_INGRESAR_EMPLEADOS DISABLE;
```

Sintaxis para habilitar un trigger que está deshabilitado:

```
ALTER TRIGGER nombredisparador ENABLE;
```

Ejemplo: Habilitamos el trigger **tr_actualizar_empleados**:

```
ALTER TRIGGER tr_actualizar_empleados ENABLE;
```

Se pueden habilitar o deshabilitar todos los trigger establecidos sobre una tabla específica, se emplea la siguiente sentencia;

```
ALTER TABLE tabla DISABLE ALL TRIGGERS;--deshabilita  
ALTER TABLE tabla ENABLE ALL TRIGGERS;-- habilita
```

La siguiente sentencia deshabilita todos los triggers de la tabla **empleados**:

```
ALTER TABLE empleados ENABLE ALL TRIGGERS;
```

Podemos saber si un trigger está o no habilitado consultando el diccionario de la Base de Datos la tabla **user_triggers**. En la columna **status** aparece **enabled** si está habilitado y **disabled** si está deshabilitado.

Eliminar un Disparador

Para eliminar un trigger se emplea la siguiente sentencia:

```
DROP TRIGGER nombrettrigger;
```

Por ejemplo:

```
DROP TRIGGER tr_insertar_libros;
```

Si eliminamos una tabla, se eliminan todos los triggers establecidos sobre ella.

Errores Definidos por el Usuario en un Trigger

Oracle permite definir a los usuarios errores especificando un número de error y un mensaje empleando el procedimiento **raise_application_error**.

Sintaxis:

```
RAISE_APPLICATION_ERROR (numero,texto);
```

El procedimiento **raise_application_error** permite emitir un mensaje de error. El **NUMERO** de mensaje debe ser un número negativo entre -20000 y -20999 y el mensaje de **TEXTO** una cadena de caracteres de hasta 2048 bytes.

Si durante la ejecución de un trigger se produce un error definido por el usuario, se anulan todas las actualizaciones realizadas por la acción del trigger así como el evento que la activó, es decir, se reanuda cualquier efecto retornando un mensaje y se deshace la orden ejecutada.

En caso de que se incluya en el cuerpo de un disparador **AFTER** (que se ejecuta después de la sentencia, es decir, cuando los datos ya han sido actualizados), la sentencia será deshecha (**rollback**). Si es un disparador **BEFORE** (que se ejecuta antes de la sentencia, o sea, cuando los datos aún no han sido actualizados), la sentencia no se ejecuta.

Veamos un ejemplo:

Crearemos un trigger de actualización a nivel de fila sobre la tabla **libros**. Ante cualquier modificación de los registros de **libros**, se debe ingresar en la tabla **control**, el nombre del usuario que realizó la actualización y la fecha; pero, controlaremos que **NO** se permita modificar el campo **codigo**, en caso de suceder, la acción no debe realizarse y debe mostrarse un mensaje de error:

```
CREATE OR REPLACE TRIGGER tr_actualizar_libros
BEFORE UPDATE
ON libros
FOR EACH ROW
BEGIN
  IF UPDATING('codigo') THEN
    RAISE_APPLICATION_ERROR(-20001,'No se puede modificar el código de los libros');
  ELSE
    INSERT INTO control VALUES(USER,SYSDATE);
  END IF;
END;
/
```

Si se actualiza cualquier campo de **libros**, se dispara el trigger; si se actualiza el campo **codigo**, aparece un mensaje de error y la actualización no se realiza; en caso de actualizarse cualquier otro campo, se almacenará en **control**, el nombre del usuario y la fecha.

Creación de un Disparador INSTEAD OF

Una vista presenta el resultado de una consulta como una tabla. Si desea modificar un campo dentro de una vista como si se tratara de una tabla, se deberá crear un disparador de tipo INSTEAD OF. Lo que hará que en lugar de modificar los datos de la vista, se modifiquen los datos de las tablas subyacentes en la vista. Por ejemplo, consideremos que tenemos la siguiente vista **emp_ubicacion**, cuya columna **nombre** se crea a partir de la concatenación de las columnas **apellido** y **nombre** de la tabla **empleados**:

```
CREATE VIEW emp_ubicacion AS
SELECT e.emple_id, e.apellido || ', ' || e.nombre nombre,
       d.nombre_depto departamento, l.ciudad ciudad, c.nombre_país país
FROM empleados e, departamentos d, ubicacion l, paises c
WHERE e.id_departamento = d.id_departamento AND
      d.id_de_ubicación = l.id_de_ubicación AND l.id_de_país = c.id_de_país
ORDER BY apellido;
```

Para actualizar con la vista **emp_ubicación** el **nombre** se debería actualizar el **apellido** y el **nombre** en la tabla de **empleados**. Esto es lo que permite hacer un disparador **INSTEAD OF**.

```
CREATE OR REPLACE TRIGGER actualizar_nombre_vista
INSTEAD OF UPDATE ON emp_ubicacion
BEGIN
  UPDATE empleados SET
    nombre = substr( :NEW.nombre, instr( :new.nombre, ',' )+2),
    apellido = substr( :NEW.nombre, 1, instr( :new.nombre, ',' )-1)
  WHERE emple_id = :OLD.emple_id;
END;
/
```

NEW y OLD son pseudoregistros que el motor de ejecución de PL/SQL crea y rellena cada vez que se activa un disparador de nivel de fila. OLD y NEW almacenan los valores original y nuevo, respectivamente, del registro que procesa el disparador. Se denominan pseudoregistros porque no poseen todas las propiedades de los registros PL/SQL.

Ejercicios de Disparadores

1. Crear un trigger que se dispare cada vez que se inserta un registro en la tabla **libros**, dejando el registro en la tabla **control** el usuario que intentó hacer la inserción del libro y la fecha del evento.
2. Crear un disparador que se ejecute cada vez que un registro se ingresa a la tabla **ofertas**, dejando registrado en la tabla **control** el usuario que quiso hacer la inserción y fecha del evento.
3. Crear un disparador que se ejecute una vez si se eliminan registro de la tabla **libros**, dejando el registro en la tabla **control** el usuario que intentó hacer la eliminación y fecha del evento.
4. Crear un desencadenador que se dispare cada vez que se actualice algún campo en la tabla **libros**, dejando registrado en la tabla **control** el usuario que intentó hacer la eliminación y fecha del evento.
5. Crear un desencadenador a nivel de fila que se dispare cada vez que se vaya a actualizar el campo **precio** de la tabla **libros**, dejando el registro en la tabla **control** el usuario que intentó hacer la eliminación y la fecha del evento.
6. Crear un trigger a nivel de fila que se dispare cada vez que se vaya a actualizar el campo **precio** de la tabla **libros**, dejando el registro en la tabla **control** el usuario que intentó hacer la actualización y la fecha del evento.
7. Crear un disparador a nivel de fila que se dispare antes de que se ejecute una actualización del precio de un libro. Registrar en la tabla control, el nombre del usuario que realizó la actualización, la fecha, el código del libro que ha sido modificado, el precio anterior y el nuevo precio.
8. Crear un trigger que controle el precio que se está actualizando, si éste supera los \$50.000, se debe tomar el 80% del nuevo valor redondeándolo al entero menor, caso contrario actualizar al valor indicado (siempre que este sea un entero, sino redondearlo). Registrar en la tabla control, el nombre del usuario que realizó la actualización, la fecha, el código del libro que ha sido modificado, el precio anterior y el nuevo precio.
9. Realizar un disparador que ante cualquier actividad de cambio sobre la tabla **departamentos** (actualizaciones, eliminaciones o inserciones de registros) registre en la tabla de AUDITORIA el usuario, la fecha y el nombre de la tabla afectada.
10. Crear una Vista que muestre un listado con los siguientes datos de cada alumno matriculados en cada curso que dictan los profesores: legajo del alumno, el apellido y el nombre separados por una coma pegada al apellido, el nombre de la materia a la que se ha matriculado y el nombre del profesor que la dicta (usar las tablas **alumnos**, **cursos**, **profesor** y **matricula**). Crear un disparador para permitir modificar el apellido y el nombre de un alumno utilizando la vista creada.